

Código Limpo Aplicado à Área de Dados

Guia técnico aprofundado sobre arquitetura, boas práticas, bibliotecas, SQL, BI e versionamento aplicados ao ecossistema de dados corporativo moderno.

A Filosofia Fundamental: Código limpo em dados vai muito além de indentação. Significa construir pipelines determinísticos, consultas eficientes, modelos semânticos robustos e análises auditáveis que evitam decisões baseadas em métricas corrompidas.

1. Estruturação de Projetos e Organização de Pastas

O maior erro inicial em projetos de dados é o uso de diretórios caóticos e arquivos avulsos sem padrão. Um projeto de dados profissional deve seguir uma estrutura modular clara, separando ingestão, processamento, modelagem e relatórios.

- **Diretório /data:** Subdividido rigorosamente em *raw* (dados brutos imutáveis), *processed* (dados intermediários limpos) e *curated/gold* (dados finais prontos para consumo analítico).
- **Diretório /src ou /scripts:** Contém os códigos modulares em Python, divididos por responsabilidade (ex: ingestão, limpeza, engenharia de features, treinamento e inferência).
- **Diretório /sql:** Centraliza todas as consultas de transformação, visões (views) e procedures executadas diretamente no Data Warehouse.
- **Diretório /reports ou /bi:** Armazena arquivos de template de dashboards, metadados de relatórios e documentação visual.

Essa separação garante que engenheiros de dados, analistas de BI e cientistas de dados trabalhem em um ambiente limpo, versionável e sem acoplamento indevido entre lógicas de negócios e arquivos locais temporários.

2. Padrões de Código Limpo em Python (Pandas, Polars & Scikit-Learn)

Bibliotecas de manipulação e modelagem de dados exigem rigor técnico para evitar códigos lentos, mutações de estado ocultas e cadernos (Jupyter Notebooks) ilegíveis.

Princípios Aplicados ao Pandas e Polars

Evite encadeamentos cegos e variáveis genéricas (como df1, df2, temp). Prefira abordagens funcionais, tipagem estática e legibilidade semântica.

```
# RUIM (Mutações ocultas, variáveis genéricas e código espaguete) df = pd.read_csv('base.csv') df = df[df['status'] == 'ATIVO'] df['receita_liq'] = df['valor'] * 0.85 df = df.dropna(subset=['cliente_id']) # BOM (Funcional, legível, tipado e com método pipe) def filtrar_e_calcular_receita(caminho: str) -> pd.DataFrame: return ( pd.read_csv(caminho) .query("status == 'ATIVO'") .assign(receita_liq = lambda x: x['valor'] * 0.85) .dropna(subset=['cliente_id']) )
```

Práticas Essenciais em Python para Dados

- **Tipagem Estática com Type Hints:** Declare os tipos de entrada e saída das funções analíticas utilizando o módulo typing.
- **Funções Puras:** Funções de transformação de dados nunca devem alterar o DataFrame original no escopo global (evite inplace=True sem necessidade).
- **Gestão de Memória:** Converta colunas de texto repetitivas para o tipo category em datasets volumosos e otimize tipos numéricos (ex: int64 para int32).
- **Modularização de Notebooks:** Utilize Jupyter apenas para exploração inicial; o código de produção deve residir em scripts Python limpos e testáveis.

3. Queries de Código e Modelagem SQL Limpa

Consultas SQL mal estruturadas geram gargalos críticos de processamento em Data Warehouses modernos (Snowflake, BigQuery, Redshift) e tornam a manutenção de pipelines inviável.

Estruturação de CTEs (Common Table Expressions)

Subconsultas aninhadas e confusas prejudicam a legibilidade. Utilize CTEs encadeadas de forma lógica e progressiva (Ingestão -> Filtros -> Agregações -> Seleção Final).

```
WITH clientes_elegiveis AS ( SELECT id_cliente, data_cadastro FROM tb_clientes WHERE status_conta = 'ATIVO' ), transacoes_ult_trimestre AS ( SELECT id_cliente, SUM(valor_transacao) AS faturamento_total FROM tb_transacoes WHERE data_transacao >= CURRENT_DATE - INTERVAL '90' DAY GROUP BY id_cliente ) SELECT c.id_cliente, COALESCE(t.faturamento_total, 0) AS faturamento_3m FROM clientes_elegiveis c LEFT JOIN transacoes_ult_trimestre t ON c.id_cliente = t.id_cliente;
```

Diretrizes Avançadas de SQL Limpo

- **Zero SELECT *** : Sempre declare explicitamente as colunas desejadas para proteger seu modelo contra alterações estruturais upstream.
- **Padronização de Nomenclatura**: Siga convenções rígidas (ex: snake_case, prefixando dimensões com dim_ e fatos com fat_ ou tb_).
- **Palavras Reservadas em Maiúsculas**: Mantenha operadores e comandos (SELECT, FROM, WHERE, JOIN, GROUP BY) em letras maiúsculas para contraste visual imediato.

4. Código Limpo na Montagem de BI e Modelagem Semântica

Ferramentas de Business Intelligence (Power BI, Tableau, Looker) possuem códigos subjacentes (DAX, Power Query / M, LookML) que exigem rigor arquitetural e governança de métricas.

Princípios de Organização em BI

- **Separação de Camadas:** Nunca realize transformações complexas na camada visual se puderem ser tratadas no Data Warehouse ou na linguagem M do Power Query.
- **Clean DAX:** Evite medidas gigantescas e monolíticas. Quebre cálculos complexos utilizando variáveis (VAR / RETURN) para otimizar a performance e a legibilidade.
- **Modelagem Dimensional (Star Schema):** Estruture sempre tabelas fato centralizadas rodeadas por tabelas dimensão limpas. Evite tabelas achatadas gigantescas que explodem o consumo de memória RAM do motor analítico.

Governança de Indicadores: Padronize o dicionário de dados corporativo. É inadmissível que relatórios diferentes apresentem fórmulas divergentes para o cálculo de "Churn" ou "Receita Líquida".

UX de Dados e Storytelling

Um dashboard limpo segue a hierarquia visual natural (Z-Pattern), contendo no máximo 4 a 5 gráficos por página, cores com significado semântico e tooltips ricas que respondem a dúvidas em segundos.

5. Git, GitHub e Documentação Rigorosa de Projetos de Dados

Código limpo perde o sentido se não estiver versionado e documentado adequadamente. Em ciência e análise de dados, o versionamento garante a reprodutibilidade de análises e modelos estatísticos.

Padrões de Versionamento (Git)

- **Commits Semânticos:** Mensagens claras que indicam o escopo da alteração (ex: `feat(etl):` adiciona tratamento de nulos ou `fix(dax):` corrige regra de faturamento YTD).
- **O que NÃO versionar (.gitignore):** Jamais envie chaves de API, credenciais de banco de dados, arquivos de cache ou datasets brutos volumosos (utilize armazenamento externo ou DVC).
- **Estratégia de Branching:** Separe ambientes de desenvolvimento, homologação e produção utilizando fluxos estruturados (GitFlow ou GitHub Flow).

A Estrutura de um README.md Profissional

Todo projeto de dados de alto padrão deve conter documentação técnica clara cobrindo:

- **Objetivo de Negócio:** Qual dor ou problema a análise ou modelo resolve?
- **Arquitetura e Linhagem de Dados:** O fluxo completo desde a origem até o destino final.
- **Guia de Execução:** Instruções passo a passo para rodar scripts e notebooks de forma reproduzível.

Conclusão: A adoção estrita de código limpo na área de dados eleva o nível técnico da organização, transformando artefatos caóticos em ativos corporativos perenes, seguros e de altíssima confiabilidade.